

PROBLEM 1: JAVA METHOD OVERRIDING

Platform: HackerRank

Topic: Method Overriding

Aim:

To understand method overriding in Java, where a child class provides its own implementation of a method inherited from the parent class.

Input:

No direct user input.

The program creates an instance of a child class and calls the overridden method.

Expected Output:

I am eating

JAVA CODE

```
class Animal {  
  
    void eat() {  
        System.out.println("I am eating");  
    }  
  
}  
  
class Dog extends Animal {  
  
    @Override  
    void eat() {  
        System.out.println("I am eating");  
    }  
  
}  
  
public class Solution {
```

```
public static void main(String[] args) {  
  
    Animal animal = new Dog();  
  
    animal.eat();  
}  
  
}
```

Output:

I am eating

Explanation:

The Dog class overrides the eat() method of the Animal class.

The method called depends on the actual object created.

Complexity:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

PROBLEM 2: JAVA METHOD OVERRIDING 2 – SUPER KEYWORD

Platform: HackerRank

Topic: Method Overriding and super Keyword

Aim:

To understand how the super keyword can be used to call a method of the parent class from the child class.

Input:

No direct user input.

Expected Output:

Hello, I am a motorcycle, I am a cycle with an engine.

My ancestor is a cycle who is a vehicle with pedals.

JAVA CODE

```
class BiCycle {

    String define_me() {

        return "a vehicle with pedals.";
    }

    BiCycle() {

        System.out.println(
            "Hello I am a bicycle."
        );
    }

}

class Motorcycle extends BiCycle {

    Motorcycle() {

        System.out.println(
            "Hello I am a motorcycle, " +
            "I am a cycle with an engine."
        );

        String temp = super.define_me();

        System.out.println(
            "My ancestor is a cycle who is " +
            temp
        );
    }

}
```

```
}  
  
}  
  
public class Solution {  
  
    public static void main(String[] args) {  
  
        Motorcycle m = new Motorcycle();  
    }  
  
}
```

Output:

Hello I am a bicycle.

Hello I am a motorcycle, I am a cycle with an engine.

My ancestor is a cycle who is a vehicle with pedals.

Explanation:

super is used to access members of the parent class.

Here, super.define_me() calls the define_me() method of BiCycle.

Complexity:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

PROBLEM 3: JAVA SINGLETON

Platform: HackerRank

Topic: Singleton Design Pattern

Aim:

To implement the Singleton Design Pattern so that only one object of a class can be created.

Input:

No direct user input.

Example:

String input = "Hello Singleton";

Output:

Hello Singleton

JAVA CODE

```
class Singleton {  
  
    private static Singleton instance;  
  
    public String str;  
  
    private Singleton() {  
    }  
  
    public static Singleton getInstance() {  
  
        if (instance == null) {  
  
            instance = new Singleton();  
        }  
  
        return instance;  
    }  
  
}  
  
public class Solution {
```

```
public static void main(String[] args) {  
  
    Singleton singleton =  
        Singleton.getInstance();  
  
    singleton.str = "Hello Singleton";  
  
    System.out.println(  
        singleton.str  
    );  
}  
  
}
```

Output:

Hello Singleton

Explanation:

The constructor is private, so objects cannot be created directly from outside the class.

The getInstance() method returns the only available instance.

Complexity:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

PROBLEM 4: JAVA VISITOR PATTERN

Platform: HackerRank

Topic: Visitor Design Pattern

Aim:

To understand the Visitor Design Pattern and perform operations on different types of nodes in a tree without changing the node classes.

Input:

The tree structure contains nodes with:

- Value
- Color
- Depth
- Children

Example:

A tree can contain:

Value: 4

Color: RED

Depth: 0

Output:

The visitor calculates the required values according to the type of visitor.

JAVA CODE

```
import java.util.*;
```

```
abstract class Tree {
```

```
    private int value;
```

```
    private Color color;
```

```
    private int depth;
```

```
    public Tree(
```

```
        int value,
```

```
        Color color,
```

```
        int depth) {
```

```
        this.value = value;
```

```

        this.color = color;
        this.depth = depth;
    }

    public int getValue() {
        return value;
    }

    public Color getColor() {
        return color;
    }

    public int getDepth() {
        return depth;
    }

    public abstract void accept(TreeVis visitor);

}

class TreeNode extends Tree {

    private ArrayList<Tree> children =
        new ArrayList<>();

    public TreeNode(
        int value,
        Color color,
        int depth) {

        super(value, color, depth);
    }

    public void accept(TreeVis visitor) {

        visitor.visitNode(this);

        for (Tree child : children) {

            child.accept(visitor);

```



```

    }
}

public void addChild(Tree child) {

    children.add(child);
}

}

class TreeLeaf extends Tree {

public TreeLeaf(
    int value,
    Color color,
    int depth) {

    super(value, color, depth);
}

public void accept(TreeVis visitor) {

    visitor.visitLeaf(this);
}

}

enum Color {
RED, GREEN
}

abstract class TreeVis {

public abstract int getResult();

public abstract void visitNode(TreeNode node);

public abstract void visitLeaf(TreeLeaf leaf);

}

```

```

class SumInLeavesVisitor extends TreeVis {

    private int result = 0;

    public void visitNode(TreeNode node) {
    }

    public void visitLeaf(TreeLeaf leaf) {

        result += leaf.getValue();
    }

    public int getResult() {

        return result;
    }

}

class ProductOfRedNodesVisitor extends TreeVis {

    private long result = 1;

    public void visitNode(TreeNode node) {

        if (node.getColor() == Color.RED) {

            result = (result * node.getValue())
                % 1000000007;
        }
    }

    public void visitLeaf(TreeLeaf leaf) {

        if (leaf.getColor() == Color.RED) {

            result = (result * leaf.getValue())
                % 1000000007;
        }
    }
}

```

```

}

public int getResult() {

    return (int) result;
}

}

class FancyVisitor extends TreeVis {

private int evenDepthSum = 0;
private int greenLeafSum = 0;

public void visitNode(TreeNode node) {

    if (node.getDepth() % 2 == 0) {

        evenDepthSum += node.getValue();
    }
}

public void visitLeaf(TreeLeaf leaf) {

    if (leaf.getColor() == Color.GREEN) {

        greenLeafSum += leaf.getValue();
    }
}

public int getResult() {

    return Math.abs(
        evenDepthSum - greenLeafSum
    );
}

}

```

Explanation:

The Visitor pattern separates the operation from the object structure.

Different visitors can perform different operations on the same tree.

Complexity:

Time Complexity: $O(n)$

Space Complexity: $O(n)$

=====

PROBLEM 5: JAVA ITERATOR

Platform: HackerRank

Topic: Iterator

Aim:

To understand how an Iterator can be used to traverse elements of a collection.

Input:

A collection containing strings and integers.

Example:

String:

"###"

"Hello"

"###"

"World"

Output:

Hello

World

JAVA CODE

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        ArrayList<String> list =
            new ArrayList<>();

        list.add("###");
        list.add("Hello");
        list.add("###");
        list.add("World");

        Iterator<String> iterator =
            list.iterator();

        while (iterator.hasNext()) {

            String value = iterator.next();

            if (!value.equals("###")) {

                System.out.println(value);
            }
        }
    }
}
```

Output:

Hello

World

Explanation:

Iterator provides a standard way to traverse elements of a collection.

Important methods:

hasNext()

next()

Complexity:

Time Complexity: $O(n)$

Space Complexity: $O(1)$

=====

PROBLEM 6: JAVA INTERFACE

Platform: HackerRank

Topic: Interface

Aim:

To understand how interfaces provide abstraction and how a class implements an interface.

Input:

$a = 2$

$b = 3$

Output:

5

JAVA CODE

```
interface AdvancedArithmetic {
```

```

int divisorSum(int n);

}

class Calculator
implements AdvancedArithmetic {

public int divisorSum(int n) {

    int sum = 0;

    for (int i = 1; i <= n; i++) {

        if (n % i == 0) {

            sum += i;

        }

    }

    return sum;

}

}

public class Solution {

public static void main(String[] args) {

    Calculator calculator =
        new Calculator();

    System.out.println(
        calculator.divisorSum(6)
    );

}

}

```

Output:

12

Explanation:

The divisors of 6 are:

1, 2, 3, 6

Their sum is:

$1 + 2 + 3 + 6 = 12$

The Calculator class implements the AdvancedArithmetic interface.

Complexity:

Time Complexity: $O(n)$

Space Complexity: $O(1)$

=====

PROBLEM 7: JAVA HASHSET

Platform: HackerRank

Topic: HashSet

Aim:

To understand how HashSet stores only unique elements and automatically removes duplicate pairs.

Input:

Number of pairs:

5

Pairs:

john tom

john mary

john tom

mary anna

mary anna

Output:

4

Explanation:

Unique pairs are:

john tom

john mary

mary anna

Therefore:

Number of unique pairs = 3

JAVA CODE

```
import java.util.*;
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int t = sc.nextInt();
```

```
HashSet<String> set =  
    new HashSet<>();  
  
for (int i = 0; i < t; i++) {  
  
    String first = sc.next();  
    String second = sc.next();  
  
    set.add(first + " " + second);  
  
    System.out.println(  
        set.size()  
    );  
}  
  
sc.close();  
}
```

Sample Input:

5

john tom

john mary

john tom

mary anna

mary anna

Sample Output:

1

2

2

3

3

Explanation:

HashSet does not allow duplicate elements.

Complexity:

Average Time Complexity per insertion: $O(1)$

Total Time Complexity: $O(n)$

Space Complexity: $O(n)$

=====

PROBLEM 8: JAVA FACTORY

Platform: HackerRank

Topic: Factory Design Pattern

Aim:

To understand the Factory Design Pattern and create objects based on the requested type.

Input:

cake

Output:

The factory creates a Cake object.

JAVA CODE

```
interface Food {
```

```
String getType();

}

class Pizza implements Food {

    public String getType() {

        return "Pizza";
    }

}

class Cake implements Food {

    public String getType() {

        return "Cake";
    }

}

class FoodFactory {

    public Food getFood(String order) {

        if (order.equalsIgnoreCase("pizza")) {

            return new Pizza();

        } else {

            return new Cake();

        }

    }

}

public class Solution {
```

```
public static void main(String[] args) {  
  
    FoodFactory factory =  
        new FoodFactory();  
  
    Food food =  
        factory.getFood("cake");  
  
    System.out.println(  
        "The factory returned class " +  
        food.getClass().getSimpleName()  
    );  
  
    System.out.println(  
        "I am eating " +  
        food.getType()  
    );  
}  
  
}
```

Output:

The factory returned class Cake

I am eating Cake

Explanation:

The FoodFactory decides which object should be created.

The calling code does not need to directly create Pizza or Cake objects.

Complexity:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

PROBLEM 9: JAVA EXCEPTION HANDLING

Platform: HackerRank

Topic: Exception Handling

Aim:

To understand how exceptions can be handled using try-catch blocks.

Input:

10
3

Output:

3

Example of invalid input:

10
0

Output:

java.lang.ArithmeticException:
/ by zero

JAVA CODE

```
import java.util.*;

public class Solution {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int a = sc.nextInt();
```

```

int b = sc.nextInt();

try {

    System.out.println(a / b);

} catch (ArithmeticException e) {

    System.out.println(
        "Exception: " + e
    );
}

sc.close();
}

}

```

Explanation:

The division operation is placed inside the try block.

If division by zero occurs, ArithmeticException is caught by the catch block.

Complexity:

Time Complexity: $O(1)$

Space Complexity: $O(1)$

=====

PROBLEM 10: JAVA EXCEPTION HANDLING – TRY CATCH

Platform: HackerRank

Topic: Try-Catch

Aim:

To handle invalid input and arithmetic errors using exception handling.

Input:

10

3

Output:

3

If the second number is zero:

Input:

10

0

Output:

java.lang.ArithmeticException:
/ by zero

JAVA CODE

```
import java.io.*;  
import java.util.*;
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        try {
```

```
            int x = sc.nextInt();
```

```
            int y = sc.nextInt();
```

```
            System.out.println(x / y);
```

```
        } catch (InputMismatchException e) {
```



```
        System.out.println(
            "java.util.InputMismatchException"
        );

    } catch (ArithmeticException e) {

        System.out.println(
            "java.lang.ArithmeticException: " +
            "/" by zero"
        );
    }

    sc.close();
}
}
```

Example 1:

Input:

10
3

Output:

3

Example 2:

Input:

10
0

Output:

java.lang.ArithmeticException: / by zero

Example 3:

10

a

Output:

`java.util.InputMismatchException`

Explanation:

try contains the code that may generate an exception.

catch handles the exception without terminating the program unexpectedly.

COMPLEXITY

Time Complexity: $O(1)$

Space Complexity: $O(1)$